3.5 定点数的加减运算——溢出检测

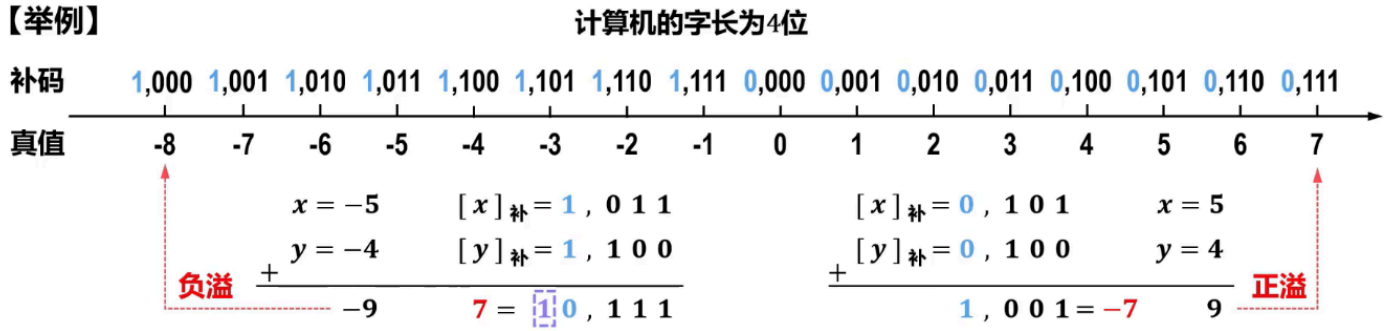
考纲定位：计算机组成原理 · 第 3 章 运算方法（3.5）｜重要度 ★★★★★｜常见考法：选择题/大题（溢出条件、三种检测法、逻辑表达式、变形补码）

一句话核心：溢出 = 结果超出补码表示范围；只有同号相加或异号相减才可能溢出，检测看“符号位是否异常”或“进位是否一致”

一、溢出的概念

溢出的概念

- **计算机的字长是有限的，因此所能表示的数据范围也是有限的。**
- **当运算结果超出所能表示的数据范围时，就会出现溢出（Overflow）。**
  - **溢出会导致错误的运算结果。**



定义

- 两个 n 位补码数运算，结果超出 $[-2^{n-1}, 2^{n-1}-1]$，称为溢出
- 溢出时结果低 n 位仍按模 2^n 保留，但数值错误

发生条件

- 同号相加：可能溢出
- 异号相加：不可能溢出
- 同号相减：不可能溢出
- 异号相减：可能溢出

注：只有两个操作数符号相同时，才可能发生溢出

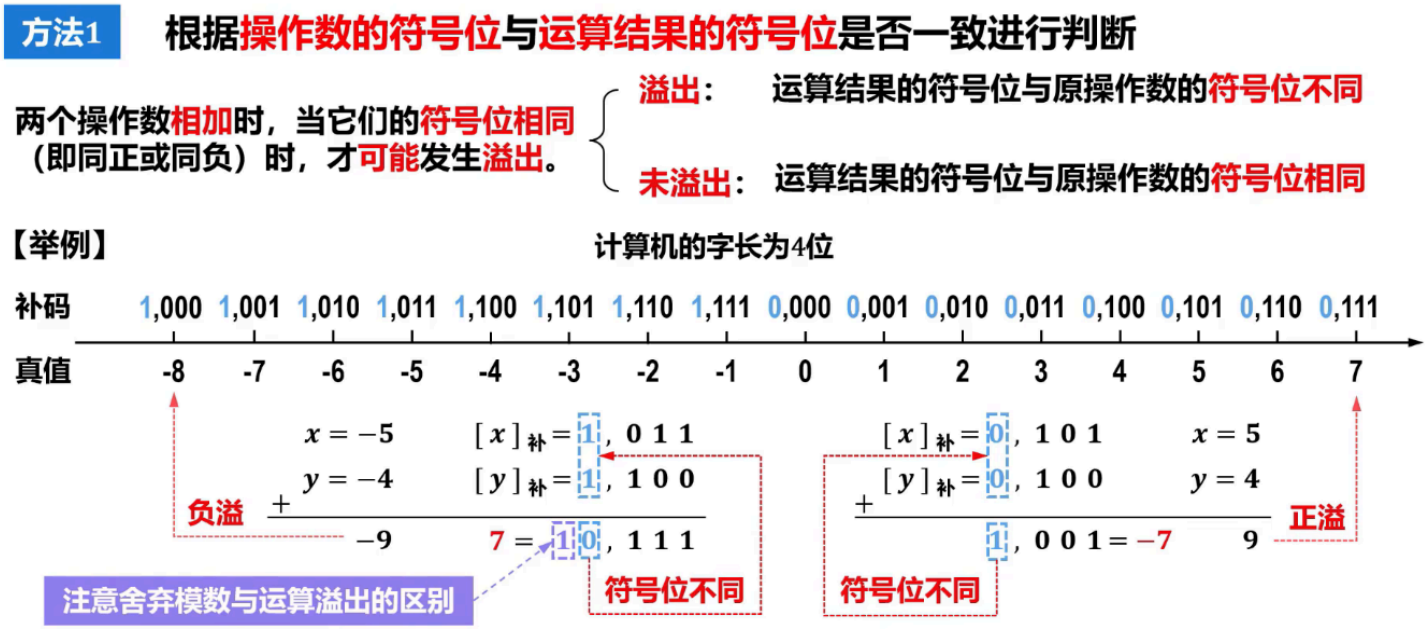
易错

- 最高位进位不等于溢出；无符号数看进位，有符号数看符号位
- 异号相加结果一定在范围内，不会溢出
- 减法先转成加法，再按“实际参与运算的操作数”判断符号

二、溢出检测的方法

2.1 法1：检测操作数与运算结果的符号位是否一致

规则



判据

- 同号相加，结果符号与操作数不同 → 溢出
- 正+正得负，或负+负得正 → 溢出

逻辑表达式

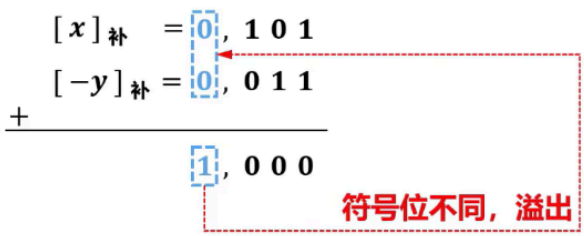
$$OF = \overline{A_s B_s} C_s + A_s B_s \overline{C_s}$$

- A_s, B_s：两个操作数的符号位
- C_s：结果符号位

注：操作数的符号指实际参与运算的操作数的符号；减法 $x - y$ 转成 $x + (-y)$，$-y$ 的符号才是需要考虑的

例题

【举例】若$x = +101$，$y = -011$，采用4位补码表示，判断$[x - y]_{补}$是否会溢出。



逻辑表达式图

操作数 x 的符号位记为 x_{n-1}，操作数 y 的符号位记为 y_{n-1}，运算结果 s 的符号位记为 s_{n-1}，
溢出标志位记为 OF，当 OF 为1时表示发生溢出。

x 与 y 的符号位相同，与 s 的符号位相反 $\begin{cases} x_{n-1}\cdot y_{n-1}\cdot \overline{s}_{n-1} \\ \overline{x}_{n-1}\cdot \overline{y}_{n-1}\cdot s_{n-1} \end{cases}$

$$OF = x_{n-1}\cdot y_{n-1}\cdot \overline{s}_{n-1} + \overline{x}_{n-1}\cdot \overline{y}_{n-1}\cdot s_{n-1}$$
$$OF = 1\cdot 1\cdot \overline{1} \;+\; \overline{1}\cdot \overline{1}\cdot 1 = 0$$

【例题1】若 $x=-101$，$y=-011$，采用4位补码表示，判断 $[x+y]_{补}$ 是否会溢出。

$$[x]_{补} = \overset{x_{n-1}}{1},\ 0\ 1\ 1$$
$$[y]_{补} = 1,\ 1\ 0\ 1$$
$$+\quad\ \ y_{n-1}$$
$$\overset{1}{1},\ 0\ 0\ 0$$
$$s_{n-1}$$

符号位相同，未溢出

符号位进位的值为模数，应舍弃

易错

- 该法只适用于“同号相加”；异号相加不能套用
- 减法要先转成加法，再判断实际操作数符号
- 结果符号位是运算后的符号位，不是操作数符号

2.2 法2：检测最高数值位的进位与符号位的进位是否一致

规则

溢出检测

方法2 根据运算过程中**最高数值位的进位**与**符号位的进位**是否一致进行判断

不同可判定为产生**溢出**，**相同**可判定为**没有产生溢出**。

【举例】 若 $x=5$，$y=4$，采用4位补码表示，判断 $[x+y]_{补}$ 是否会溢出。

符号位的进位 ← 0≠1 → 最高数值位的进位

进位

$x=5$ $[x]_{补}$ 0，1 0 1

$y=4$ $[y]_{补}$ 0，1 0 0

+

9 正溢 1，0 0 1 = −7

【举例】 若 $x=-5$，$y=-4$，采用4位补码表示，判断 $[x+y]_{补}$ 是否会溢出。

符号位的进位 ← 1≠0 → 最高数值位的进位

进位

$x=-5$ $[x]_{补}$ 1，0 1 1

$y=-4$ $[y]_{补}$ 1，1 0 0

+

负溢 −9 → 1 0，1 1 1 = 7

符号位进位的值为模数，应舍弃

判据

$$OF = C_n \oplus C_{n-1}$$

- C_n：符号位向外的进位
- C_{n-1}：最高数值位向符号位的进位
- 两者不同 → 溢出

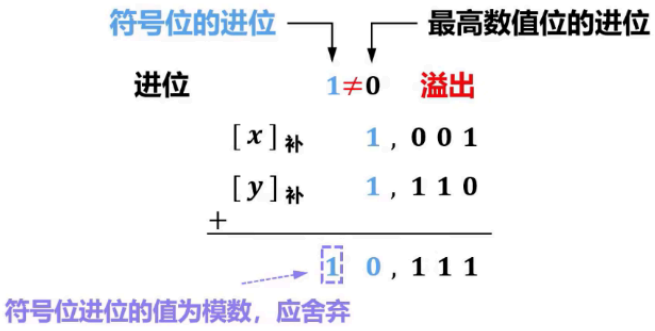
例题

溢出检测

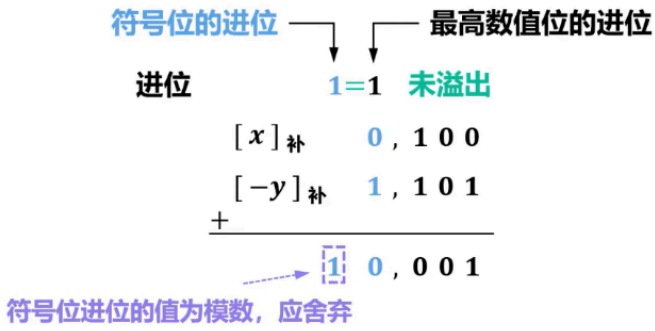
方法2 根据运算过程中**最高数值位的进位**与**符号位的进位**是否一致进行判断

不同可判定为产生**溢出**，**相同**可判定为**没有产生溢出**。

【练习3】若 $x=-111$，$y=-010$，采用4位补码表示，判断 $[x+y]_{补}$ 是否会溢出。



【练习4】若 $x=+100$，$y=+011$，采用4位补码表示，判断 $[x-y]_{补}$ 是否会溢出。



逻辑表达式

溢出检测

方法2 根据运算过程中**最高数值位的进位**与**符号位的进位**是否一致进行判断

不同可判定为产生**溢出**，**相同**可判定为**没有产生溢出**。

最高数值位的进位记为 C_{n-1}，符号位的进位记为 C_n

溢出标志位记为 OF，当 OF 为1时表示发生溢出。

C_{n-1} 与 C_n 不同，则产生溢出（即 OF 为1）：

写出**溢出检测**的**逻辑表达式**？

$$OF = C_{n-1} \oplus C_n$$

易错

- C_n 是“符号位产生的进位”，不是最高数值位进位
- 两者相同即无溢出，不同即溢出
- 该法对加法和减法都适用（减法已转加法）

2.3 法3：变形补码（具有两位符号位）检测法

规则

溢出检测

方法3 **利用变形补码（具有2位符号位的补码）的符号位进行判断**

变形补码也称为双符号补码，具有2个符号位，其余与补码相同。

- 双符号位为 00 时，表示正数；
- 双符号位为 11 时，表示负数；
- 双符号位为 01 时，表示正溢出；
- 双符号位为 10 时，表示负溢出；

溢出判断非常直观，适合手工运算时的溢出检测，但其硬件成本高，在计算机中主要采用单符号溢出检测方案。

注意：负数变形补码转换成原码仍可以使用反码法或扫描法。

判据

- 双符号位 `00`：结果为正，正常
- 双符号位 `11`：结果为负，正常
- 双符号位 `01`：正溢出
- 双符号位 `10`：负溢出

逻辑表达式

$$OF = S_{s1} \oplus S_{s2}$$

- 两个符号位不同 → 溢出
- 高位符号位表示“真正的符号”，低位符号位参与运算

例题

溢出检测

方法3 **利用变形补码（具有2位符号位的补码）的符号位进行判断**

- 双符号位为 00 时，表示正数；
- 双符号位为 11 时，表示负数；
- 双符号位为 01 时，表示正溢出；
- 双符号位为 10 时，表示负溢出；

【练习】(1) $[x]_{补}=00.1011$，$[y]_{补}=00.0111$，判断 $[x]_{补}+[y]_{补}$ 的结果是否溢出。

(2) $[x]_{补}=11.0101$，$[y]_{补}=11.0011$，判断 $[x]_{补}+[y]_{补}$ 的结果是否溢出。

(3) $[x]_{补}=00.1011$，$[y]_{补}=11.0011$，判断 $[x]_{补}+[y]_{补}$ 的结果是否溢出。

(1)

$$\begin{array}{rl} & [x]_{补} \quad 00.1011 \\ + & [y]_{补} \quad 00.0111 \\ \hline & [x]_{补}+[y]_{补} \quad \boxed{01}.0010 \end{array}$$

正溢出

(2)

$$\begin{array}{rl} & [x]_{补} \quad 11.0101 \\ + & [y]_{补} \quad 11.0011 \\ \hline & [x]_{补}+[y]_{补} \quad \boxed{10}.1000 \end{array}$$

负溢出

(3)

$$\begin{array}{rl} & [x]_{补} \quad 00.1011 \\ + & [y]_{补} \quad 11.0011 \\ \hline & [x]_{补}+[y]_{补} \quad \boxed{11}.1110 \end{array}$$

未溢出

逻辑表达式图

溢出检测

方法3 **利用变形补码（具有2位符号位的补码）的符号位进行判断**

- 双符号位为 00 时，表示正数；
- 双符号位为 01 时，表示正溢出；
- 双符号位为 11 时，表示负数；
- 双符号位为 10 时，表示负溢出；

写出溢出检测的逻辑表达式?



左起第一个符号位记为C_{n+1}

左起第二个符号位记为C_n

溢出标志位记为OF，当OF为1时表示发生溢出。

C_{n+1} 与 C_n 不同，则产生溢出（即OF为1）：

$$OF = C_{n+1} \oplus C_n$$

易错

- 双符号位不是“两个独立的符号”，而是把符号位扩展为两位
- `01` 是正溢出，`10` 是负溢出，不要记反
- 变形补码运算后，最高符号位才是结果的真符号

三、三种方法对比

方法	判据	优点	局限
符号位法	同号相加，结果异号	直观	只适用于同号相加
进位法	$C_n \oplus C_{n-1}$	硬件简单，加法器直接可用	要区分两个进位
变形补码	双符号位不同	判断直观，便于硬件	需要额外符号位

统一结论

- 溢出只发生在“同号相加/异号相减”
- 硬件最常用 $OF = C_n \oplus C_{n-1}$
- 变形补码的 $OF = S_{s1} \oplus S_{s2}$ 本质相同

知识关联：补码加减（3.4 定点数的加减运算——补码运算规则）；串行加法器（3.8 串行进位加法器的硬件逻辑实现）；CF/OF（5.8 指令的操作类型）